MTGEDH Oracle IR Architecture & Execution Loop

Version: 2026-06-12 (batch 279) | **Repository:** MTGEDH | **Audience:** Contributors working on card behavior and server integration

1. Purpose

Oracle IR (Intermediate Representation) is the canonical pipeline for turning **oracle text** into **game state changes**. The end goal is a single path:

```
oracle text → parse → OracleEffectStep[] → execute → server sync → resolution UI → resume
```

Layers 1–2 (parse + dry-run execute) are complete for Commander scope. Layer 3 (live server integration) is partial: production runs IR-primary, but legacy fallbacks still run when IR returns incomplete.

2. Three Layers

Layer	Location	Responsibility	Status
1 — Parse	`rules-engine/src/oracleIRParser.ts`	`parseOracleTextToIR()` → steps	Green (~30k cards)
2 — Execute	`rules-engine/src/oracleIRExecutor.ts`	`applyOracleIRStepsToGameState()`	Green (triggers 100% assisted)
3 — Integrate	`server/src/state/modules/oracleIr*.ts`	Live state, queue, resume	**Partial**

> **Rule:** Never call `applyOracleIRStepsToGameState` on live server state. Always use `applyOracleIRStepsOnServer`.

3. Core Data Types

OracleEffectStep

Typed instructions in `rules-engine/src/oracleIR.ts`: `draw`, `mill`, `create_token`, `add_counter`, `tap_or_untap`, `conditional`, `choose_mode`, `unknown` (gap — must not return `handled: true`).

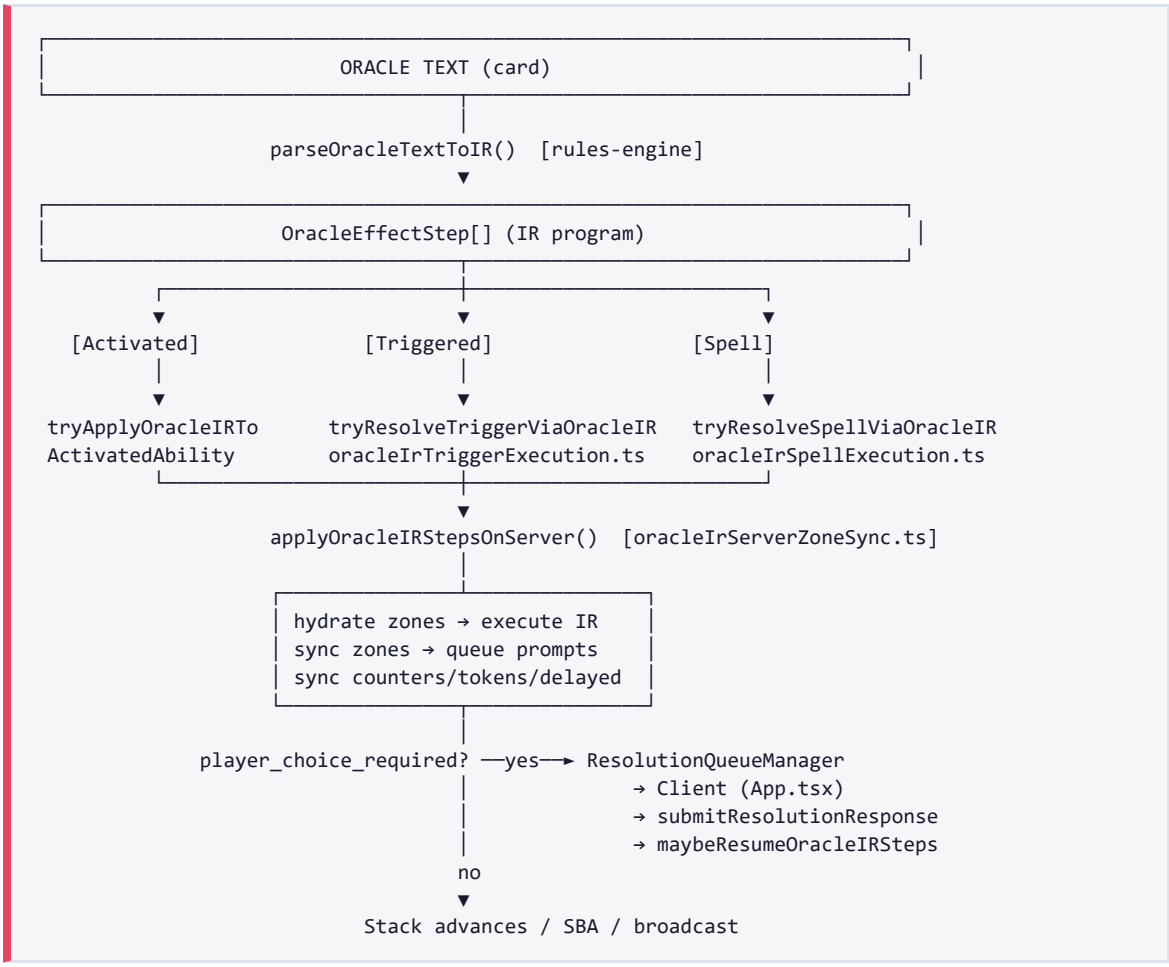
OracleIRExecutionContext

Built via `buildOracleIRExecutionContext()`: controller, source, targets, `chosenObjectIds`, `xValue`.

OracleIRExecutionResult

`appliedSteps`, `skippedSteps` (with reason codes), `pendingOptionalSteps`, `pendingContinuationSteps`, mutated `state`.

4. Architecture Diagram



5. Production Feature Flags

File: `server/src/state/modules/oracleIrFeatureFlags.ts`

Flag	Production	Meaning
`ORACLE_IR_TRIGGER_PRIMARY`	**ON**	Try IR before trigger regex
`ORACLE_IR_SPELL_PRIMARY`	**ON**	Try IR before categorizeSpell
`ORACLE_IR_*_LEGACY=1`	off	Force legacy path

Vitest defaults primary **OFF** (`server/tests/vitest.setup.ts`). Migration suite tests production defaults.

6. Execution Loop (Phases)

Phase A — Stack entry

1. **Activated** → `tryApplyOracleIRToActivatedAbility` (`oracleIrAbilityExecution.ts`)
2. **Triggered** → `tryResolveTriggerViaOracleIR` (`oracleIrTriggerExecution.ts`)
3. **Spell** → `tryResolveSpellViaOracleIR` (`oracleIrSpellExecution.ts`)

Each: extract effect text → `parseOracleTextToIR` → collect steps → check `oracleIrStepWiring.ts` runtime wiring.

Phase B — All-or-nothing gate

Invariant: Multi-step effects must complete mandatory steps or return `handled: false`. Partial IR + legacy fallback = double tokens/effects (primary bug class).

Phase C — Server apply

`applyOracleIRStepsOnServer` in `oracleIrServerZoneSync.ts`:

1. `hydrateOracleIRPlayerZonesFromContext`
2. `applyOracleIRStepsToGameState` with `haltOnPlayerChoiceRequired: true`
3. Sync zones, counters, tokens, delayed flicker returns
4. `queue*ChoicePromptsFromOracleIRResult` per paused step kind

Phase D — Resolution queue bridge

IR step	Resolution type
`scry`, `surveil`	`SCRY` / `SURVEIL`
`unless_pays_mana/life`	`UNLESS_PAY`
`discard`	`DISCARD_SELECTION`

`sacrifice`	`TARGET_SELECTION`
`look_select_top`	`LIBRARY_SEARCH`
`add_mana`	`MANA_COLOR_SELECTION`
`player_choice`	`OPTION_CHOICE`
`copy_permanent`	`TARGET_SELECTION`
`fateseal`, `clash`	Effect Program → `PLAYER_CHOICE`

Client: `resolutionStepPrompt` → modal in `App.tsx` → `submitResolutionResponse` → `maybeResumeOracleIRStepsAfterResolution`.

Phase E — Effect Program

For multi-choice chains: `rules-engine/src/effectProgram.ts`, `startOracleEffectProgramOnServer`, `resumeEffectProgramResolution`. Production for optional trigger yes/no and fateseal/clash; **not yet default** for modal `choose_mode` spells.

Phase F — Legacy fallback (migration debt)

Path	File
Trigger regex	`oracleIrTriggerLegacy*.ts` (31 modules)
Spell categorize	`targeting.ts` → `executeSpellEffect`
Uncategorized spell	`oracleIrUncategorizedSpellFallback.ts`
Trigger IR tail	`stackExecuteTriggerEffectIrFallbackUtils.ts`

7. Ability-Type Loops

Triggered abilities

```
Game event → detect*Triggers() → stack (APNAP)
  → tryResolveTriggerViaOracleIR()
       ├─ Effect Program? → startOracleEffectProgramOnServer
       ├─ applyOracleIRStepsOnServer()
       └─ handled? done : executeTriggerEffect() legacy
  → resolution prompts → resume → broadcast
```

Tap batching: `collectTapTriggersForTapEvent()` — "one or more X become tapped" fires once per event.

Activated abilities

```
interaction.ts → costs paid → stack
  → tryApplyOracleIRToActivatedAbility()
       → collectActivatedAbilitySteps() (text after "{cost}:")
       → applyOracleIRStepsOnServer()
  → queue + resume
```

Most complete IR integration path.

Spells

```
stack → tryResolveSpellViaOracleIR() [SPELL_PRIMARY]
  → handled? done
  → else categorizeSpell() → executeSpellEffect()
  → else oracleIrUncategorizedSpellFallback
```

8. Key Files

Concern	Path
Parse	`rules-engine/src/oracleIRParser.ts`
IR types	`rules-engine/src/oracleIR.ts`
Execute	`rules-engine/src/oracleIRExecutor.ts`
Server apply + queue	`server/src/state/modules/oracleIrServerZoneSync.ts`
All-or-nothing wiring	`server/src/state/modules/oracleIrStepWiring.ts`
Trigger entry	`server/src/state/modules/oracleIrTriggerExecution.ts`
Spell entry	`server/src/state/modules/oracleIrSpellExecution.ts`
Activated entry	`server/src/state/modules/oracleIrAbilityExecution.ts`

Resolution socket	`server/src/socket/resolution.ts`
Feature flags	`server/src/state/modules/oracleIrFeatureFlags.ts`

9. Adding New Behavior

1. **Parser** — `oracleIRParser*.ts` + rules-engine test
2. **Executor** — `oracleIRExecutor*StepHandlers.ts`
3. **Wiring** — `trigger-runtime-wiring.ts` step kind sets
4. **Server queue** — `queue*FromOracleIRResult` + `resolution.ts` resume
5. **Integration test** — `server/tests/oracle-ir-*.ts` with PRIMARY=1
6. **Retire** — legacy regex if superseded

> **Forbidden:** card-name routing, new `KNOWN_*` table rows, bespoke `executeTriggerEffect` branches. See `.cursor/rules/oracle-ir-no-bespoke-handlers.mdc`.

10. Verification

```
cd server
npm run test:oracle-ir-migration    # 649 tests, production defaults
npm run trigger:coverage
npm run executor:coverage
```